

Apuntes sobre Redes Neuronales

Modelos de la computación II Grupos A y B

Javier Jiménez Dorador
Alfonso García Gallardo
María del Carmen Montalbán Fernández
Víctor Manuel Cruz Dueñas

Índice

<u>1.Introducción</u>	<u>3</u>
<i>1.1 Forma clásica de resolver un sistema: imitación de la naturaleza</i>	3
<i>1.2 ¿Cómo abordar la construcción de una red neuronal con un computador?</i>	4
<i>1.3 Aplicaciones de las redes neuronales</i>	6
<u>2.Redes neuronales de una capa</u>	<u>7</u>
<i>2.1 Células de McCulloch-Pitts</i>	7
<i>2.2 Perceptrón</i>	8
<i>2.3 Adaline</i>	9
<u>3.Redes neuronales multicapa</u>	<u>11</u>
<i>3.1 Perceptrón multicapa</i>	11
<i>3.2 Redes neuronales de base radial</i>	13
<i>3.3 Redes neuronales recurrentes</i>	14
<u>4.Técnicas de validación</u>	<u>15</u>
<u>Bibliografía</u>	<u>17</u>

1. Introducción

1.1 Forma clásica de resolver un problema: imitación de la naturaleza

Desde el principio de los tiempos, la raza humana ha intentado imitar a la naturaleza cada vez que se le presentaba un problema, hoy en día, debido al avance de la técnica los problemas que se nos plantean son mucho más complejos, pero no por ello el entorno natural que nos rodea deja de ser una importante fuente de inspiración.

La forma de inteligencia más compleja que conocemos es el cerebro humano, por ello, sería interesante conocer sus mecanismos y funcionamiento para tratar de imitarlo mediante máquinas.

Haciendo una simplificación, el cerebro es un conjunto de neuronas enlazadas unas con otras formando una red, cada neurona posee un conjunto de dendritas que recogen información de las neuronas de su entorno, un núcleo donde se procesa esa información y un axón que propaga la respuesta por la red.

El método por el cual se transmiten información las neuronas es llamado sinapsis, y todavía no se entiende completamente, pero a un nivel básico se puede decir que las neuronas transmiten estímulos excitadores o inhibidores, si el estímulo recibido sobrepasa cierto umbral la neurona envía una señal de activación, este concepto es importante para el posterior estudio de las redes neuronales, las neuronas no son elementos lineales.

1.2 ¿Cómo abordar la construcción de una red neuronal con un computador?

Para acometer la tarea de representar una red neuronal con un sistema de computación deben hacerse unas consideraciones iniciales:

a) Las redes neuronales biológicas se caracterizan por tener un numero gigantesco de neuronas todas ellas trabajando en paralelo, hoy por hoy, este paralelismo masivo no se puede acometer en ningún tipo de computadora.

b) Las conexiones entre una neurona y otra se hace de forma electroquímica, es decir, las señales son analógicas, los sistemas en los que modelaremos las redes neuronales son digitales.

Para representar una neurona, podemos considerar que las dendritas serán las entradas al núcleo y el axón la salida, cada entrada al núcleo (al igual que en una neurona biológica) tendrá una importancia, un pequeño cambio en una dendrita puede tener mas influencia que un gran cambio en el entorno de otra, por ello, es necesario dotar a unas entradas de mayor peso que otras en el sistema.

Imitando a la realidad, modelaremos el núcleo de tal forma que frente a determinados conjuntos de entradas active su salida, mientras que para otros la salida sera inhibidora o nula.

Como herramienta matemática que surgió, esta idea de neurona es fácil de representar de forma analítica:

a) Cada entrada y salida es un valor numérico.

b) Los pesos de cada entrada son una característica propia de la neurona que asignará mas importancia a unas que a otras, clásicamente esto se resuelve multiplicando la entrada por el peso asignado a la conexión.

c) La activación o no de la neurona se evaluará mediante una función matemática que depende del diseñador de la red.

Puede parecer a simple vista que en este modelo se ha pasado por alto el poder estimulación de la neurona (excitación/inhibición) sin embargo esto no es cierto, ya que una conexión inhibitoria es representada mediante un peso negativo para dicha conexión.

De esta forma, ya poseemos una neurona fácilmente computable, para formar una red neuronal completa es necesario instanciar un conjunto de neuronas y conectarlas mediante alguna topología concreta para intentar resolver el problema para la que fue diseñada.

1.3 Aplicaciones de las redes neuronales

A lo largo del tema se comprobará como a partir de una simple red neuronal se pueden resolver problemas muy complejos de una forma cómoda, rápida y efectiva.

Una red neuronal artificial es capaz de aprender determinados comportamientos mediante un conjunto de ejemplos, esto es muy útil para problemas para los que el algoritmo que los resuelve es ineficiente, difícilmente implementable o incluso no se conoce.

Actualmente, las redes neuronales se usan en multitud de campos, y es centro de muchas investigaciones, por lo que se puede decir que es una tecnología en pleno auge, algunos campos en los que sobresalen es en el control de algunos componentes robóticos, en la elaboración de diagnósticos tanto médicos como de ingeniería, en predicciones económicas y meteorológicas, generalizaciones simples de funciones matemáticas complejas o reconocimiento de patrones.

2. Redes neuronales de una capa

2.1 Células de McCulloch-Pitts

Se puede considerar que la primera aproximación a la modelización de una neurona la llevaron a cabo McCulloch y Pitts en 1943, fueron las llamadas células de McCulloch-Pitts.

Estas células asignan un peso a cada una de las conexiones (W_i), por cada conexión llegaba un estímulo o entrada (X_i) siendo el comportamiento de la célula el siguiente:

- a) 1 si $\sum X_i * W_i > \theta$
- b) 0 en caso contrario

A θ se le conoce como umbral o bias y es un valor que elige el diseñador de la red.

Aunque seguramente la mejor forma de modelar una red neuronal no sea mediante células de McCulloch-Pitts, fueron la primera aproximación y además poseen la capacidad de computación universal, es decir, cualquier programa computable mediante un modelo de cómputo lo suficientemente potente, puede ser representado por esta estructura. La demostración del apartado anterior se basa en comprobar que mediante estas células se pueden caracterizar las funciones lógicas AND, OR y NOT que constituyen la infraestructura de un computador. Si se elige, 1 entrada con peso -1 y bias -1 tendremos la función NOT, dos entradas con pesos 1 y bias 1 la función AND, y dos entradas con pesos 1 y bias 0, obtendremos la función OR.

En contra de esta estructura se puede decir, que una red formada por células McCulloch-Pitts, cuando se enfrenta a un problema complejo, no es fácilmente tratable, ya que el número de células necesarias y la gran cantidad de conexiones necesarias entre ellas, hacen muy difícil de usar en la práctica.

2.2 Perceptrón

Este modelo se concibió como un método capaz de realizar tareas de clasificación de forma automática, la idea era disponer de un sistema que, a partir de un conjunto de ejemplos de clases diferentes, fuera capaz de distinguir la frontera entre ellas.

La estructura de un perceptrón es la siguiente: un conjunto de células de entrada y otro de células de salida, ambos según convenga para el problema a resolver y cada una de las células de entrada conectada con todas las células de salida, y un umbral (θ).

La salida del perceptrón se calcula de la siguiente manera: $f(\sum W_i * X_i, \theta)$

Y $f(x,y)$:

1 si $x > y$

-1 en caso contrario

El perceptrón, por sí mismo es capaz de clasificar categorías binarias linealmente separables, es trivial demostrar que al igual que las células de McCulloch-Pitts, los perceptrones poseen capacidad de computación universal.

Un resultado muy interesante del trabajo con perceptrones es el llamado teorema de convergencia del perceptrón, el enunciado de dicho teorema es el siguiente: “Si hay una configuración de pesos que clasifica correctamente una serie de ejemplos de entrenamiento, entonces el algoritmo de aprendizaje los encontrara en un número finito de iteraciones”.

El mayor problema del perceptrón es que solo puede separar conjuntos linealmente separables, lo cual era de esperar dada su simplicidad, además, al ser sus salidas binarias, solo pueden codificar un conjunto discreto de estados.

2.3 Adaline

El Adaline surgió de la idea de que si un perceptrón tuviera como salidas a los números reales, se podría codificar con él cualquier salida, y estos nuevos perceptrones (Adaline) se podrían convertir en sistemas de resolución general de problemas.

Una ventaja de este método sobre el perceptrón es que mientras que el anterior modelo aprendía básicamente por refuerzo (es decir, se potencian las salidas correctas y no se tienen en cuenta las incorrectas, no se tiene en cuenta el grado de error de la salida producida, en cambio, en este modelo sí. El aprendizaje incorpora una nueva característica de aprendizaje, la regla Delta, o el valor absoluto de la diferencia entre la salida esperada y la producida.

El error total de la red se calcula mediante el error cuadrático medio entre las salidas producidas y esperadas, como esta es una medida global del error de la red, el algoritmo de aprendizaje tratará de minimizarla, para ello, se usa la técnica de descenso en gradiente.

De la forma más intuitiva posible se intentará explicar el algoritmo de aprendizaje de este tipo de neuronas, la técnica de descenso en gradiente, que trata de disminuir la función de error siguiendo la línea de máxima pendiente descendente, aunque es una técnica razonablemente buena, sigue presentando algunos inconvenientes.

En este algoritmo aparece un nuevo parámetro de la red, la tasa de aprendizaje (alfa), se puede decir, que es la velocidad a la que variará el estado de la red, o desde otro punto de vista, es una constante proporcional a la amplitud del intervalo que avanzará en el descenso en gradiente. Elegir una correcta tasa de aprendizaje es decisivo para el éxito o no de la red, una tasa de aprendizaje elevada, hace que la red aprenda muy rápidamente el problema, pero puede ocasionar que cuando se aproxime a un mínimo de la función, la red tome alternativamente valores a la izquierda y derecha del pico. Por contra, una tasa de aprendizaje pequeña no produce estos efectos secundarios, pero la red aprende muy lentamente, una técnica muy útil a utilizar es comenzar el entrenamiento de la red con una tasa de aprendizaje elevada e ir disminuyendo esta conforme se va alcanzando una configuración cercana a la solución.

Aunque la salida de una neurona Adaline es un número real, también podemos usar estas neuronas como clasificadores al igual que usábamos los perceptrones, para esto tan solo hay que binarizar su salida, es decir, considerar un umbral y a partir de él distinguir dos clases.

Con todo esto puede parecer que esta estructura junto con la técnica de descenso en gradiente es ideal para resolver todo tipo de problemas, sin embargo, como se verá en desarrollos posteriores, el descenso en gradiente presenta más problemas aparte de los anteriormente mencionados.

3. Redes neuronales multicapa

3.1 Perceptrón multicapa

Aunque las neuronas artificiales pueden resolver algunos tipos de problemas, su potencial se hace manifiesto cuando se combinan gran cantidad de estas entidades formando una red, hubo varios intentos de llevar a cabo esta tarea, pero no fue hasta 1986 cuando se encontró un algoritmo de entrenamiento útil para este tipo de redes.

Algunos autores [Cybenko, 1989], [Hornik et al. 1989] demostraron que el perceptrón multicapa es capaz de aproximar cualquier función continua sobre un espacio compacto de R^n .

La capacidad de los perceptrones multicapa para resolver problemas reales los ha hecho muy populares, sin embargo, también tienen sus limitaciones, para gran número de variables de entrada y salida se hace muy difícil su aprendizaje, incluso en problemas en los que debido al desconocimiento del número de variables que influyen pueden añadirse algunas que no determinen las salidas y también hagan inmanejable el problema.

La arquitectura típica de un perceptrón multicapa es: una capa de entrada, una de salida y varias ocultas, conectividad total y los enlaces todos hacia adelante, cada conexión con un peso asociado.

El número de capas ocultas y de nodos en cada una de ellas, así como la función de activación de cada neurona, son elegidas por el diseñador de la red según conveniencia. Las funciones de activación más usadas en este tipo de redes son la sigmoideal y la tangente hiperbólica, no hay gran diferencia en usar cualquiera de ellas puesto que son combinación lineal una de la otra, su forma está ampliamente retratada en la bibliografía, pero para nuestros propósitos nos basta con saber que se asemejan bastante a la función escalón.

A la hora de diseñar nuestra red con un perceptrón multicapa hay que tener en cuenta la complejidad del problema que vamos a abordar, aunque no hay que dejarse llevar por la falsa impresión que un número mayor de nodos conduce siempre a una mejor solución, el único medio para alcanzar una buena productividad con redes neuronales es usar una serie de heurísticas que sólo la experiencia en el campo nos puede dar.

El algoritmo usado para llevar a cabo el aprendizaje de los perceptrones multicapa se llama retropropagación y es un algoritmo de aprendizaje supervisado, para ello utiliza el descenso en gradiente, esto es debido en gran parte a la no linealidad de las funciones de activación de los nodos. A parte de las ventajas y problemas apuntados anteriormente, el método de descenso en gradiente presenta algunos problemas más, como puede ser la parálisis de la red o el estancamiento en mínimos locales. Para intentar dar mayor estabilidad al algoritmo de aprendizaje se introdujo un nuevo parámetro, el momentum o inercia, este parámetro hace que tenga un peso relativo el desplazamiento anterior de la red.

Los problemas de análisis pueden ser resueltos en parte con algunas heurísticas, si una red sufre parálisis, suele ocurrir cuando los parámetros de la red toman valores muy grandes, por ello, se puede evitar iniciando los pesos en torno a cero. El hecho de caer continuamente en mínimos locales puede ser debido a una escasa capacidad de representación de la red, por eso la solución sería aumentar el número de neuronas ocultas.

Los pasos que componen el proceso de aprendizaje del perceptrón multicapa son los siguientes:

- 1) Se inicializan los pesos y umbrales de la red, generalmente se hace aleatoriamente y con valores cercanos a cero.
- 2) Se toma un patrón de entrada y se propaga hasta la salida, obteniéndose así la respuesta de la red.
- 3) Se evalúa el error cuadrático cometido por la red.
- 4) Se modifican los pesos según el algoritmo de aprendizaje.
- 5) Se repiten los pasos 2, 3 y 4 para todos los patrones de entrada.
- 6) Se evalúa el error total cometido por la red.
- 7) Se repiten los pasos 2, 3, 4, 5 y 6 hasta alcanzar un mínimo del error del entrenamiento.

3.2 Redes neuronales de base radial

Este tipo de redes neuronales, al igual que las anteriores, tiene la propiedad de computabilidad universal, su estructura es muy similar a la del perceptrón multicapa, son redes en las que cada nodo está completamente conectado con los de la capa posterior y no está conectado con la capa anterior, además se une la restricción de poseer tan sólo una capa oculta.

Lo que realmente caracteriza a las redes neuronales de base radial es que la función de activación de las neuronas de la capa intermedia es una función gaussiana, otra característica a tener en cuenta es que en este tipo de redes las conexiones de la capa de entrada a la capa oculta carecen de peso, sin embargo, de la capa oculta a la de salida las conexiones si son ponderadas.

Las funciones gaussianas tienen forma de campana, a grandes rasgos se pueden considerar que permanece activas (o con salida distinta de cero), este comportamiento tiene como consecuencia cada neurona de la capa oculta se active para un determinado rango de entradas a la red.

Debido a las propiedades de las funciones de activación de estas redes neuronales se puede decir que en la capa oculta se divide el problema que se quiere resolver en varios subproblemas, este echo abre un nuevo campo de investigación en el campo de las redes neuronales.

3.3 Redes neuronales recurrentes

Debido a que el campo de las redes neuronales recurrentes es muy amplio, en este capítulo se va a tratar de introducir el concepto de red neuronal recurrente y se explicará los fundamentos de las redes de Elman.

La diferencia entre las redes neuronales anteriormente vistas con las recurrentes es que en estas últimas se permiten conexiones entre neuronas sin límite, en ellas si se pueden crear ciclos o bucles. Esta característica hace que las redes neuronales recurrentes tengan un poder de representación mucho mayor que las no recurrentes, sin embargo se complican mucho los algoritmos para llevar a cabo el aprendizaje de las red.

Concretamente, las redes de Elman tienen una estructura como la que sigue:

- Una capa de entrada totalmente conectada a la capa oculta.
- La capa oculta totalmente conectada a la capa de salida.
- La neuronas de la capa oculta conectadas una a una a otra capa llamada capa de contexto, y esta a su vez totalmente conectada a la capa oculta.

Se puede decir que la capa de contexto tiene exactamente los mismos valores en sus conexiones que la capa oculta pero retrasados en un unidad de tiempo, ello provoca que la salida de la red no dependa tan solo de las entradas, sino que también depende del estado anterior y por tanto se puede decir que se dota a la red neuronal de cierta memoria.

Las redes de Elman tienen la ventaja principal de que se pueden entrenar mediante el algoritmo de retropropagación, esto es así porque las conexiones entre la capa oculta y la capa de contexto son fijas, y las conexiones entre la capa de contexto y la capa oculta se pueden considerar como simples conexiones de entrada.

4. Técnicas de validación

La validación es una parte fundamental dentro del entrenamiento de una red neuronal para la resolución de un problema, esto es debido a que los patrones elegidos para el entrenamiento no tienen por qué representar exactamente como debieran al problema o incluso porque la red haya memorizado los datos siendo incapaz de generalizar la función buscada.

Cuando se comienza a entrenar una red con un conjunto de ejemplos, la red cada vez disminuye más el error cometido hasta alcanzar una cota aceptable (en caso de poseer capacidad de representación suficiente), una vez llegado a este punto se puede seguir bajando el error con entrenamiento, pero ello no nos asegura mejores rendimientos al usarla en casos reales. Esto ocurre porque aunque al principio la red va aproximándose cada vez más a la función objetivo, una vez alcanzada cierta cota al seguir entrenando la red se va particularizando cada vez más a los ejemplos produciéndose un “efecto memoria”.

Como al entrenar una red tan sólo tenemos como criterio para seguir entrenando o no el error que comete red sobre los ejemplos, no es posible detectar el efecto anterior, por ellos es necesario incluir nueva información para tener una mejor referencia sobre en qué punto del entrenamiento nos encontramos. Para situarnos más concisamente en el marco del entrenamiento una técnica habitual es separar el conjunto de ejemplos en dos subconjuntos, el de entrenamiento y el de validación, el primero se usará para entrenar directamente la red, y el segundo para comprobar la capacidad de generalización de esta, ya que el conjunto de validación solo se usa para comprobar la adecuación de la red, no para el entrenamiento.

En la mayoría de los casos al aplicar validación al entrenamiento de la red podemos obtener una medida clara de cuando la red se está sobre-entrenando o cuando generaliza de forma adecuada.

Hay otras técnicas mas sofisticadas de validación, una de ellas es la validación cruzada, se dividen los datos en un número de

conjuntos y se entrena con ellos dejando en cada iteración uno de ellos para validación, al final en la última iteración se usan todos los datos para entrenar, es un buen método aunque presenta algunos inconvenientes como que al final, en la última iteración no se usa validación.

Bibliografía

- Redes de neuronas artificiales. Un enfoque práctico.
Pedro Isasi Viñuela e Inés M. Galván León.
- Neural networks. A comprehensive foundation.
Simon Haykin.
- Apuntes de Modelos de la computación II.
Juan Luis Castro.